

Multi-Armed Bandits for Adaptive Retrieval

Group 41: Zhang Zhenjie

e0957630@u.nus.edu

A0252042W

Abstract

Traditional Retrieval-Augmented Generation (RAG) systems employ static retrieval strategies that fail to adapt to varying query complexities, leading to inefficiencies where simple queries incur unnecessary computational overhead while complex queries receive insufficient retrieval effort. This study investigates the application of Multi-Armed Bandit (MAB) algorithms to enable adaptive retrieval strategy selection based on online learning from reward feedback. Three MAB algorithms — Epsilon-Greedy, Upper Confidence Bound (UCB), and Thompson Sampling — are evaluated on the MS MARCO and Natural Questions datasets using five retrieval strategies (BM25, dense retrieval, and three hybrid search variants). A dynamic reward function balancing Recall@10 and normalized latency is employed to guide algorithm adaptation. Results demonstrate that Thompson Sampling achieves the best performance-latency trade-off, attaining 92.87% recall with 10.80ms average latency on MS MARCO, closely approximating the best static baseline (Hybrid $\alpha = 0.75$: 93.07% recall, 12.71ms latency) while maintaining greater efficiency. Ablation studies reveal that lower exploration parameters and reduced latency penalties yield higher recall at the cost of increased latency, with Thompson Sampling exhibiting superior robustness across hyperparameter configurations. The source code is available at GitHub.

1 Introduction

Retrieval-Augmented Generation (RAG) [Lewis et al., 2021] enhances large language models (LLMs) by grounding their outputs in external knowledge sources. Despite its effectiveness, standard RAG pipelines typically rely on static retrieval strategies, such as always selecting the top- k results from dense vector search. This one-size-fits-all approach often proves inefficient and suboptimal across diverse query types. Simple factual queries (e.g., “When was the Eiffel Tower built?”) can typically be resolved using lightweight sparse retrieval methods such as BM25, which incur minimal computational cost. Conversely, complex or multi-faceted queries (e.g., “Compare the economic policies of the last two U.S. presidents”) generally require more sophisticated and computationally expensive retrieval strategies, including dense retrieval or hybrid techniques that combine multiple approaches.

These observations motivate the need for adaptive retrieval strategies that dynamically adjust based on query characteristics while balancing retrieval effectiveness and computational cost. One promising direction involves formulating retrieval-strategy selection as a reinforcement learning problem, specifically using the Multi-Armed Bandit (MAB) framework. This approach follows the methodology proposed in MBA-RAG [Tang et al., 2025], which introduces cost-aware adaptive retrieval through bandit-based optimization.

This study adopts this perspective and investigates the application of Multi-Armed Bandit algorithms for adaptive retrieval, with particular focus on the retrieval component rather than the complete RAG pipeline. This retrieval-centric approach enables fine-grained examination of how different bandit algorithms balance retrieval performance and computational latency. The primary contributions of this work include: (1) implementation and evaluation of three MAB algorithms (Epsilon-Greedy, UCB, Thompson Sampling) for adaptive retrieval, (2) comprehensive analysis across two large-scale datasets (MS MARCO and Natural Questions), and (3) detailed ablation studies examining the effects of hyperparameters and reward function configurations on algorithm behavior.

2 Related Works

Recent efforts to enable adaptive retrieval have produced several notable approaches. Adaptive-RAG [Jeong et al., 2024] introduces a classifier-based routing mechanism that selects among retrieval strategies

based on predicted query complexity. The system employs a trained classifier to categorize queries and route them to appropriate retrieval methods, from simple parametric generation to complex multi-step retrieval processes.

However, MBA-RAG [Tang et al., 2025] identifies fundamental limitations in classifier-based approaches. Such methods rely on single-label supervision and heuristic cost assumptions, implicitly treating one retrieval strategy as optimal for each query despite the possibility that multiple strategies may successfully yield correct answers. This rigidity constrains exploration of alternative, potentially more robust strategies and may produce suboptimal cost-accuracy trade-offs.

To address these limitations, MBA-RAG reframes retrieval-strategy selection as a Multi-Armed Bandit problem. The bandit formulation introduces flexibility, explicit cost-awareness, and partial-feedback learning, enabling the system to explore and evaluate multiple retrieval strategies through reward-based feedback rather than rigid labels. This perspective forms the conceptual foundation for the present investigation.

Unlike MBA-RAG, which evaluates retrieval decisions within a complete RAG pipeline including generation quality assessment, this study isolates and focuses exclusively on the retrieval component. This narrower scope facilitates more precise analysis of how different multi-armed bandit algorithms influence retrieval quality, latency, and overall reward dynamics without confounding effects from downstream generation processes.

3 Methodology

3.1 Datasets

This study employs two large-scale information retrieval datasets: MS MARCO [Bajaj et al., 2016] and Natural Questions (NQ) [Kwiatkowski et al., 2019]. MS MARCO comprises anonymized Bing queries paired with web passages and human-written answers, while NQ contains real Google search queries paired with Wikipedia passages annotated by humans. Both datasets are widely adopted in retrieval and question answering research. Selection of these datasets was motivated by their scale and the fact that MBA-RAG did not utilize them, enabling independent validation of the proposed approach.

For NQ, the BEIR-processed variant [Thakur et al., 2021] is employed due to the considerably more complex structure of the original dataset, which specifies relevant evidence as HTML pages with token-level span annotations. The BEIR version simplifies preprocessing by providing direct mappings between queries and relevant passages.

Both datasets contain a corpus of passages, a set of queries, and mappings from queries to their relevant passages. However, the mappings lack relevance rankings, precluding use of rank-based metrics such as Exact Match (EM), Mean Reciprocal Rank (MRR), or nDCG within the reward function. MS MARCO contains 8,841,823 passages, 101,093 queries, and 59,273 query-relevant passage mappings, while NQ contains 2,681,468 passages, 3,452 queries, and 4,201 mappings. Notably, in MS MARCO the number of queries exceeds the number of mappings, indicating presence of queries without labeled relevant passages, necessitating additional preprocessing.

3.2 Data Preprocessing

3.2.1 Data Validation and Cleaning

Preprocessing begins with exploratory analysis to identify inconsistencies such as queries lacking corresponding relevant passage annotations or passage identifiers referenced in mappings but absent from the passage corpus. Both datasets exhibit complete alignment wherein all passage identifiers in mappings exist in the passage corpus. However, MS MARCO includes queries without relevance information; consequently, only queries with at least one annotated relevant passage are retained.

3.2.2 Corpus Reduction

The full passage collections in both datasets are extremely large, rendering direct ingestion into the retrieval database computationally prohibitive. To mitigate this constraint, a reduced corpus is constructed for each dataset consisting of all relevant passages supplemented with random samples of non-relevant passages to preserve retrieval difficulty. This yields a filtered MS MARCO subset containing 209,096 passages, 55,578 queries, and 59,273 mappings, and an NQ subset containing 200,201 passages, 3,452 queries, and 4,201 mappings.

3.2.3 Relevance Statistics

Analysis of relevance annotations indicates maximum counts of 6 relevant passages per query for MS MARCO and 4 for NQ. These values inform the selection of $k = 10$ for top- k retrieval, ensuring sufficient coverage to capture all relevant documents.

3.2.4 Embedding Precomputation

The retrieval system is implemented using locally hosted Weaviate database deployed via Docker. Weaviate is selected due to its native support for sparse (BM25), dense, and hybrid retrieval methods. Passage embeddings are precomputed prior to ingestion to leverage GPU parallelization, which is not feasible when embeddings are computed during ingestion. The embedding model employed is `multi-qa-MiniLM-L6-cos-v1`, a sentence transformer producing 384-dimensional vectors. This model is chosen for its pretraining on 215 million question-answer pairs, rendering it well-suited for semantic search applications.

During ingestion, BM25 parameters are configured to commonly employed values of $k_1 = 1.2$ and $b = 0.75$. These parameters must be fixed at ingestion time and cannot be modified subsequently.

3.2.5 Online Learning Paradigm

Unlike traditional supervised learning pipelines that split datasets into training, validation, and test subsets for offline evaluation, Multi-Armed Bandit algorithms operate in an online learning setting. In this paradigm, each incoming query simultaneously serves as both a training signal to update the bandit policy and a test instance on which the policy’s current decision is evaluated. Consequently, no train-validation-test split is applied; instead, the algorithms iteratively learn from and adapt to the sequence of queries as they are processed.

3.3 Multi-Armed Bandit Framework

3.3.1 Formal Problem Definition

The Multi-Armed Bandit problem derives its name from the colloquial term “one-armed bandit” for slot machines. The canonical analogy involves a gambler facing multiple slot machines, each with an unknown but fixed probability distribution of payouts. The objective is to determine which machines to play across sequential trials to maximize cumulative reward over time.

Formally, MAB algorithms aim to maximize total cumulative reward through strategic balancing of exploration (sampling different arms to gather information) and exploitation (selecting the currently best-performing arm). The optimal policy involves arm selection over a sequence of trials to achieve the highest possible total reward.

3.3.2 Exploration-Exploitation Trade-off

The fundamental challenge in MAB problems is the exploration-exploitation dilemma. At each timestep, the agent must choose between exploitation — selecting the arm currently believed optimal based on observed rewards to maximize immediate gain — and exploration — selecting alternative arms, possibly poorly-performing or untested ones, to gather additional information and improve reward estimates. This trade-off is central to all MAB algorithms.

3.3.3 True and Estimated Expected Rewards

The true value of an action a , denoted $q_*(a)$ or μ_a , represents the actual average reward obtained if that arm were pulled indefinitely. It is defined as the expected value of reward R_t given that action a was selected at time t :

$$q_*(a) = \mathbb{E}[R_t \mid A_t = a] \tag{1}$$

where $\mathbb{E}[\cdot]$ denotes expectation, R_t is the reward at timestep t , and A_t represents the chosen action. The goal of MAB algorithms is to learn these true values and consistently select the arm with highest $q_*(a)$.

Since true expected rewards are unknown, they must be estimated from observed rewards. The sample-average estimate for action a at time t is:

$$Q_t(a) = \frac{1}{N_t(a)} \sum_{i=1}^{t-1} R_i \cdot \mathbb{I}_{A_i=a} \quad (2)$$

where $N_t(a)$ denotes the number of times action a has been chosen by time t , and $\mathbb{I}_{A_i=a}$ is an indicator function.

For computational efficiency, the incremental update formula is employed in MAB algorithms in practice:

$$Q_{t+1}(a) = Q_t(a) + \frac{1}{N_t(a)} (R_t - Q_t(a)) \quad (3)$$

This formulation updates the estimated reward by a step size (which decreases with pull count) in the direction of the error (the difference between the received reward and the current estimate). The decreasing step size stabilizes updates as training progresses and ensures convergence. This update formula bears similarity to gradient descent in neural network training, with the error term replaced by the gradient vector and an adaptive learning rate.

3.3.4 Dynamic Reward Function

The reward function chosen for this project balances retrieval performance and latency according to:

$$R_t = A(y, \hat{y}_a) - \lambda \cdot C_{\text{norm}}(a) \quad (4)$$

where $A(y, \hat{y}_a)$ represents an accuracy metric (e.g., F1-score, Precision, or Recall), $C_{\text{norm}}(a)$ denotes normalized cost (latency), and λ controls the cost-performance trade-off. Higher λ values impose greater penalties for high-latency arms.

For the accuracy metric, Recall@10 is selected, measuring the fraction of relevant passages appearing in the top-10 retrieved results. The choice of $k = 10$ is motivated by the maximum relevant passage counts (6 for MS MARCO, 4 for NQ) identified in subsubsection 3.2.3, ensuring sufficient retrieval depth. Precision@10 is not employed because each query has varying numbers of relevant passages; even when all relevant passages are retrieved, the score may not be maximal if the retrieved set exceeds the relevant set size. F1-score, being the harmonic mean of precision and recall, would inherit this limitation.

3.4 Multi-Armed Bandit Algorithms

Three prominent MAB algorithms are implemented: Epsilon-Greedy, Upper Confidence Bound (UCB), and Thompson Sampling. Each algorithm employs distinct strategies for balancing exploration and exploitation.

3.5 Epsilon-Greedy

Epsilon-Greedy is a straightforward algorithm that balances exploration and exploitation through a fixed probability. At each timestep, it exploits current knowledge with probability $1 - \epsilon$ by selecting the arm with highest estimated average reward (greedy choice). With probability ϵ , it explores by uniformly randomly selecting any arm from the available set.

The algorithm maintains two key parameters: $Q(a)$, the estimated average reward for each arm a , and $N(a)$, the count of times each arm has been pulled. The hyperparameter $\epsilon \in [0, 1]$ controls exploration probability. At each timestep t , arm selection follows:

$$A_t = \begin{cases} \arg \max_a Q_t(a) & \text{with probability } 1 - \epsilon \quad (\text{Exploitation}) \\ \text{random arm} & \text{with probability } \epsilon \quad (\text{Exploration}) \end{cases}$$

After receiving reward R_t for arm $A_t = a$, the algorithm updates:

$$\begin{aligned} N_t(a) &\leftarrow N_t(a) + 1 \\ Q_{t+1}(a) &\leftarrow Q_t(a) + \frac{1}{N_t(a)} (R_t - Q_t(a)) \end{aligned}$$

The primary strength of Epsilon-Greedy is simplicity and guaranteed asymptotic optimality through continuous exploration. However, exploration remains uniform and does not diminish over time, potentially leading to unnecessary sampling of clearly suboptimal arms.

More details can be found at Algorithm 1 and GitHub.

3.6 Upper Confidence Bound (UCB)

UCB implements the principle of “optimism in the face of uncertainty” through deterministic exploration. Rather than random exploration, UCB calculates a score for every arm combining current average reward (exploitation) and an uncertainty term (exploration). The uncertainty term is larger for arms pulled fewer times, effectively boosting their scores and encouraging selection.

UCB maintains parameters $Q(a)$, $N(a)$, and total timestep count t . The hyperparameter $c > 0$ controls exploration strength; higher c encourages more exploration. After ensuring each arm is pulled at least once, arm selection at timestep t maximizes the UCB score:

$$A_t = \arg \max_a \left(Q_t(a) + c \sqrt{\frac{\ln(t)}{N_t(a)}} \right)$$

The term $Q_t(a)$ represents exploitation while $c \sqrt{\frac{\ln(t)}{N_t(a)}}$ represents exploration (uncertainty). Update rules follow the same incremental mean formula as Epsilon-Greedy.

UCB’s advantage lies in its principled exploration strategy that naturally decreases uncertainty as arms are sampled more, leading to more efficient convergence. The algorithm provides theoretical regret bounds, guaranteeing logarithmic cumulative regret in the best case.

More details can be found at Algorithm 2 and GitHub.

3.7 Thompson Sampling

Thompson Sampling employs a Bayesian approach, maintaining a full probability distribution (belief) about the true mean reward μ_a for each arm rather than just point estimates. For continuous rewards, this belief is modeled as a Gaussian distribution $\mathcal{N}(\mu_a, \sigma_a^2)$. This implementation constitutes Empirical Gaussian Thompson Sampling which is adopted for this project.

At each timestep, the algorithm: (1) draws one random sample from each arm’s current belief distribution, and (2) selects the arm producing the highest sample. This naturally balances exploration and exploitation: arms with high mean (high μ_a) likely produce high samples (exploitation), while arms with high variance (high σ_a^2) sometimes produce very high samples (exploration).

Thompson Sampling maintains $Q(a)$ (empirical mean), $N(a)$ (pull count), and $\text{SumSq}(a)$ (sum of squared rewards for variance calculation). The algorithm requires no exploration hyperparameter. At each timestep t , for each arm a :

1. Calculate empirical reward variance: $\hat{\sigma}_a^2 = \left(\frac{\text{SumSq}(a)}{N(a)} \right) - Q(a)^2$ (or $\hat{\sigma}_a^2 = 1.0$ if $N(a) \leq 1$)
2. Calculate standard error of the mean: $\text{SE}_a = \sqrt{\frac{\hat{\sigma}_a^2}{N(a)}}$ (or $\text{SE}_a = 1.0$ if $N(a) \leq 1$)
3. Draw sample: $S_a \sim \mathcal{N}(Q(a), \text{SE}_a^2)$
4. Select arm: $A_t = \arg \max_a (S_a)$

After receiving reward R_t for arm $A_t = a$:

$$\begin{aligned} N_t(a) &\leftarrow N_t(a) + 1 \\ \text{SumSq}_t(a) &\leftarrow \text{SumSq}_{t-1}(a) + R_t^2 \\ Q_{t+1}(a) &\leftarrow Q_t(a) + \frac{1}{N_t(a)} (R_t - Q_t(a)) \end{aligned}$$

Thompson Sampling’s primary advantage is parameter-free operation and empirically strong performance across diverse settings. The probabilistic framework naturally adapts exploration based on observed variance, typically converging efficiently.

More details can be found at Algorithm 3 and GitHub.

4 Experiments

The experimental framework follows the pipeline illustrated in Figure 1. Datasets undergo preprocessing (subsection 3.2), passages are ingested into Weaviate database, queries are streamed to MAB algorithms, algorithms select arms, arms retrieve passages from Weaviate, retrieved passages are compared with ground truth, rewards are computed via the reward function, and reward signals update MAB algorithm policies.

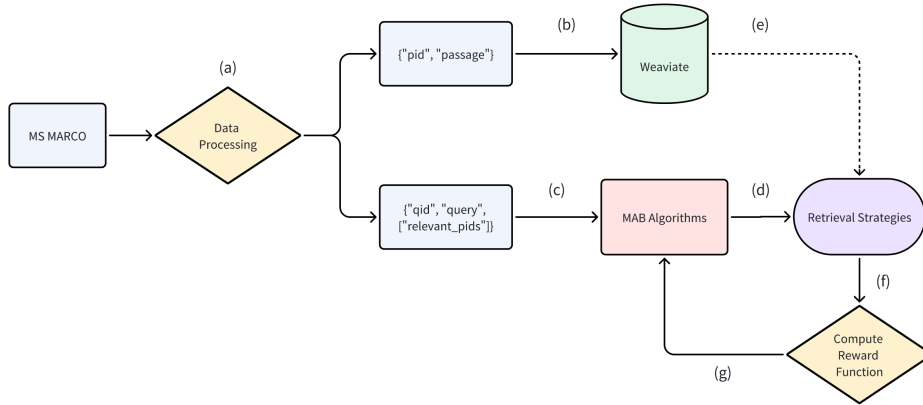


Figure 1: Experimental pipeline: (a) preprocessing, (b) database ingestion, (c) query streaming, (d) arm selection, (e) retrieval, (f) reward computation, (g) policy update.

The arms of the MAB algorithms correspond to different retrieval methods: BM25, dense retrieval using `multi-qa-MiniLM-L6-cos-v1`, and hybrid search with $\alpha \in \{0.25, 0.5, 0.75\}$. These arms represent diverse retrieval strategies with varying performance and latency characteristics. Hybrid search combines keyword search (sparse, BM25) and semantic search (dense/vector), controlled by parameter α . High α values emphasize semantic search while low α values emphasize keyword search. Importantly, both search types execute and their scores are combined to produce final rankings, providing robustness.

Baseline hyperparameters are established: the trade-off parameter λ in the dynamic reward function (Equation 4) is set to $\lambda = 0.5$. For Epsilon-Greedy, the baseline is $\epsilon = 0.1$, a commonly employed value. For UCB, the baseline is $c = 2.0$, also standard in the literature. Thompson Sampling has no hyperparameters. These baseline values are assumed unless specified otherwise.

4.1 Comparison with Static Baselines

This experiment compares MAB algorithms with static retrieval methods (the five arms individually used for all queries). All three algorithms are trained on both datasets, and 500-step moving average rewards are plotted over time to visualize learning dynamics while reducing noise as individual rewards may be noisy. Histogram visualizations show final policy proportions the MAB algorithms have settled on — the percentage of times each arm was selected by the bandit over the entire experiment, representing the learned distribution of how often each arm should be chosen over the long run to maximize total rewards. Average recall and latency at run completion are computed as well to assess performance.

Results are presented in Figure 2 and Table 1. Discussion follows in subsection 6.1.

4.2 Ablation Study: Hyperparameter Sensitivity

This experiment uses the baseline configuration but varies hyperparameters for Epsilon-Greedy and UCB to study effects on learning dynamics, final policy proportions, recall scores, and latency. Thompson Sampling is excluded due to lack of hyperparameters.

For Epsilon-Greedy, values $\epsilon \in \{0.01, 0.1, 0.3\}$ are tested. For UCB, values $c \in \{1.0, 2.0, 3.0\}$ are tested.

Results are presented in Figure 3, Figure 4, and Table 2. Discussion follows in subsection 6.2.

4.3 Ablation Study: Tradeoff Parameter Sensitivity

This experiment uses the baseline configuration but varies the reward function’s trade-off parameter λ to observe effects on learning dynamics, final policy proportions, recall scores, and latency. Different λ values alter the reward range; thus, plotting moving average rewards over time would not be meaningful. Rewards are not scaled to $[0, 1]$ to preserve interpretability (e.g., $\lambda = 2$ means normalized latency cost is twice as important as recall score).

For all three algorithms, values $\lambda \in \{0.1, 0.5, 0.9\}$ are tested.

Results are presented in Figure 5 and Table 3. Discussion follows in subsection 6.3.

4.4 Error Analysis

Error analysis is conducted only on baseline-configured MAB algorithms. Violin plots of recall score distributions for each arm of each algorithm are generated. These plots help to reveal poor decisions and arm misuse by the algorithms. Violin width indicates density at particular recall scores; wider regions denote more data points. Violin plots essentially display distribution curves for recall scores of each arm, oriented vertically.

Results are presented in Figure 6 and discussed in subsection 6.4.

5 Results



Figure 2: Multi-Armed Bandit performance across datasets: (a-b) MS MARCO and (c-d) Natural Questions. Top row shows moving average of rewards, bottom row shows learned policy proportions.

Method	MS MARCO		Natural Questions (NQ)	
	Recall@10	Latency (ms)	Recall@10	Latency (ms)
Static Baselines				
BM25	0.6961	2.28	0.5970	3.52
Dense	0.9134	7.03	0.7816	6.81
Hybrid ($\alpha = 0.25$)	0.8004	12.71	0.6813	13.07
Hybrid ($\alpha = 0.5$)	0.9206	12.68	0.7865	13.23
Hybrid ($\alpha = 0.75$)	0.9307	12.71	0.8018	13.26
Multi-Armed Bandits ($\lambda = 0.5$)				
Epsilon-Greedy ($\epsilon = 0.1$)	0.9052	6.90	0.7715	6.73
UCB ($c = 2.0$)	0.9092	8.74	0.7529	8.20
Thompson Sampling	0.9287	10.80	0.7849	9.65

Table 1: Comparison of static baselines and MAB algorithms. Metrics: Recall@10 and Latency (ms).

Algorithm	Param	MS MARCO		Natural Questions (NQ)	
		Recall@10	Latency (ms)	Recall@10	Latency (ms)
Epsilon-Greedy (ϵ)	0.01	0.9117	7.09	0.7755	7.34
	0.10	0.9052	6.90	0.7715	6.73
	0.30	0.8933	7.49	0.7642	7.37
UCB (c)	1.0	0.9204	9.52	0.7757	9.35
	2.0	0.9092	8.74	0.7530	8.20
	3.0	0.9061	7.52	0.7358	8.86

Table 2: Hyperparameter sensitivity for Epsilon-Greedy (ϵ) and UCB (c) with $\lambda = 0.5$.

6 Discussion

6.1 Comparison with Static Baselines

Examination of the 500-step moving average rewards in Figure 2 reveals that Thompson Sampling converges most rapidly and maintains the highest sustained reward levels across both datasets. This superior convergence behavior can be attributed to Thompson Sampling’s probabilistic exploration mechanism, which naturally balances exploration and exploitation through Bayesian uncertainty quantification rather than fixed exploration rates.

The policy proportion distributions in Figure 2 demonstrate distinct algorithmic preferences. Thompson Sampling predominantly favors Hybrid Search with $\alpha = 0.75$, which combines the semantic understanding of dense retrieval with keyword matching capabilities. This preference aligns with the high recall performance of this retrieval strategy. In contrast, Epsilon-Greedy and UCB exhibit stronger preferences for pure dense retrieval, suggesting these algorithms may be more sensitive to the latency penalty in the reward function, causing them to avoid the higher-latency hybrid approaches.

Analysis of Table 1 confirms expected static baseline behavior: BM25 achieves lowest recall (69.61% on MS MARCO, 59.70% on NQ) but fastest retrieval (2.28ms and 3.52ms respectively), while Hybrid Search with $\alpha = 0.75$ attains highest recall (93.07% and 80.18%) at the cost of highest latency (12.71ms and 13.26ms). This establishes the performance-latency Pareto frontier.

Among MAB algorithms, Thompson Sampling achieves the most favorable trade-off, attaining 92.87% recall with 10.80ms latency on MS MARCO and 78.49% recall with 9.65ms latency on NQ. Notably, Thompson Sampling approaches the recall of Hybrid ($\alpha = 0.75$) while reducing latency by approximately 15%, demonstrating effective adaptive strategy selection. Epsilon-Greedy exhibits the lowest latency among MAB algorithms (6.90ms and 6.73ms) but sacrifices recall (90.52% and 77.15%), indicating excessive preference for faster arms due to uniform exploration. UCB demonstrates intermediate performance but incurs higher latency than Epsilon-Greedy without proportional recall gains, suggesting less efficient exploration in this setting.

The performance-latency trade-off is comprehensively visualized in Figure 7, which plots all experi-

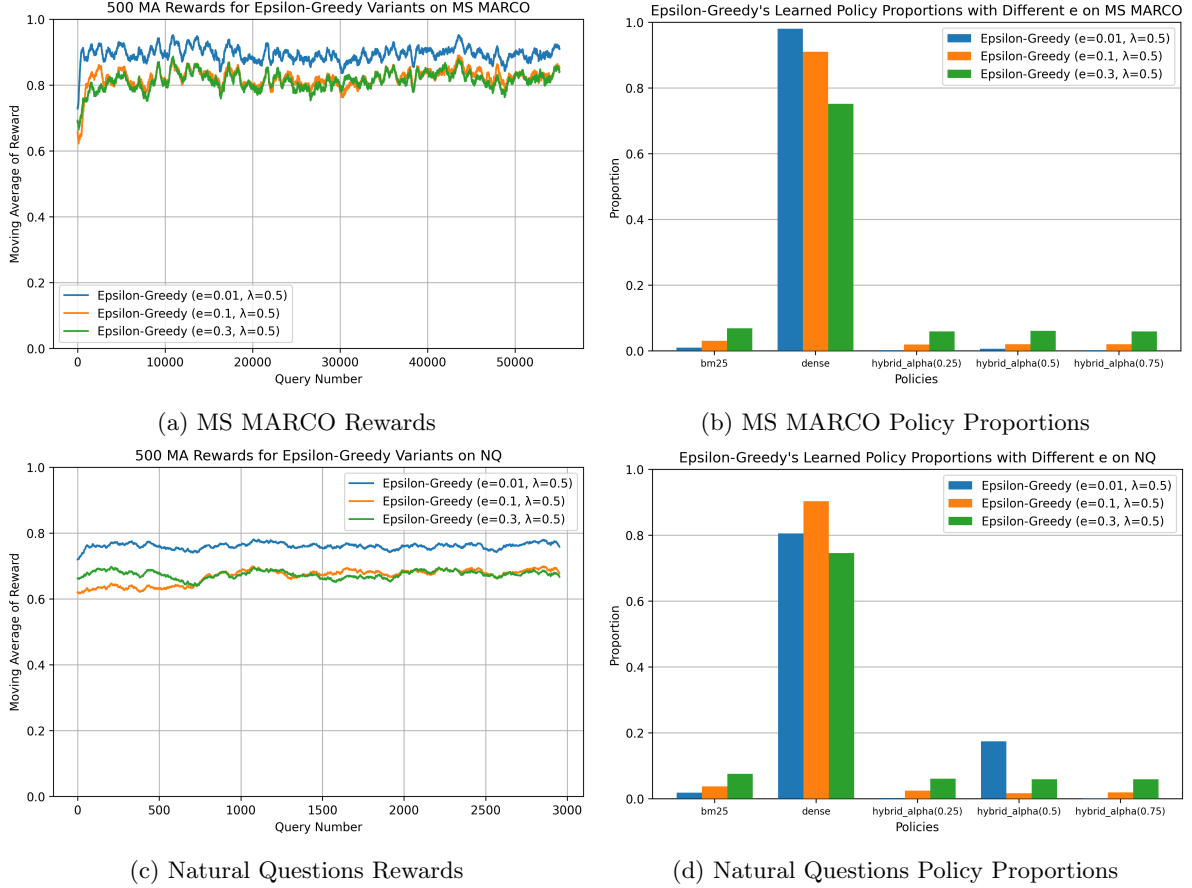


Figure 3: Epsilon-Greedy hyperparameter sensitivity across datasets: (a-b) MS MARCO and (c-d) Natural Questions.

mental configurations in recall-latency space.

6.2 Ablation Study: Hyperparameter Sensitivity

The hyperparameter sensitivity analysis for Epsilon-Greedy (Figure 3 and Table 2) reveals an inverse relationship between exploration rate ϵ and both reward and recall performance. Reducing ϵ from 0.1 to 0.01 increases average recall from 90.52% to 91.17% on MS MARCO, though with slight latency increase from 6.90ms to 7.09ms. This phenomenon occurs because lower ϵ values enable more exploitation of the best-performing arms after initial exploration, reducing wasteful sampling of suboptimal arms. The 500-step moving average rewards show substantially higher and more stable values for $\epsilon = 0.01$, indicating that once the algorithm identifies superior arms, excessive continued exploration (higher ϵ) yields diminishing returns. However, the final policy proportions remain relatively stable across ϵ values, with dense retrieval dominating, suggesting that the algorithm identifies the same strong performers regardless of exploration rate but benefits from exploiting them more frequently.

For UCB, the hyperparameter sensitivity analysis (Figure 4 and Table 2) exhibits similar trends. Reducing the exploration parameter c from 3.0 to 1.0 increases recall from 90.61% to 92.04% on MS MARCO while increasing latency from 7.52ms to 9.52ms. Lower c values reduce the uncertainty bonus in the UCB score calculation, causing the algorithm to exploit high-performing arms more aggressively. The 500-step moving average rewards demonstrate that $c = 1.0$ produces higher sustained rewards after initial learning. Interestingly, UCB’s policy proportions exhibit greater diversity across arms compared to Epsilon-Greedy, with multiple arms receiving non-negligible selection probabilities. This reflects UCB’s deterministic exploration strategy, which systematically samples less-pulled arms based on uncertainty rather than uniform randomness. However, this broader exploration does not translate to superior performance compared to Thompson Sampling.

The consistent pattern across both algorithms — lower exploration parameters yielding higher recall with modest latency increases — suggests that in this retrieval setting, the optimal arms can be identified

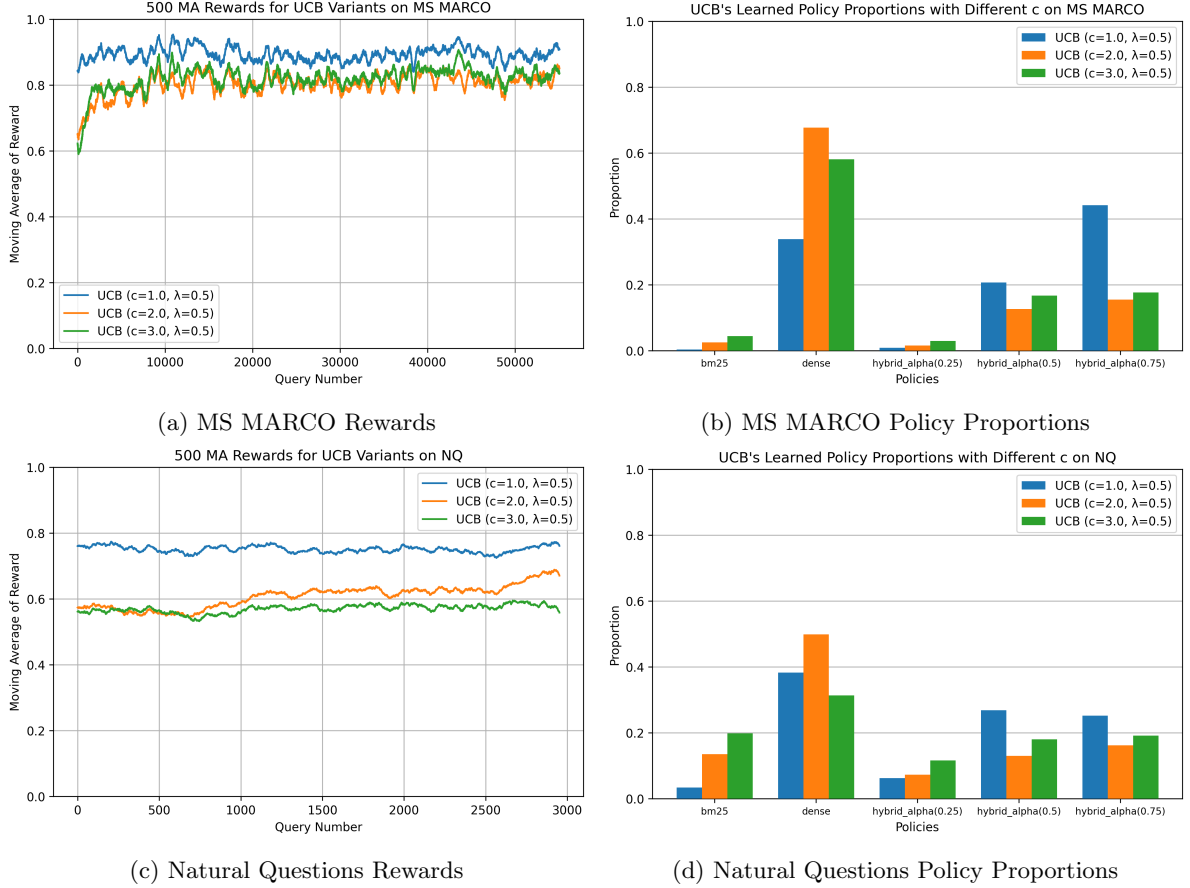


Figure 4: UCB hyperparameter sensitivity across datasets: (a-b) MS MARCO and (c-d) Natural Questions.

relatively quickly, and continued aggressive exploration after initial learning phases provides limited benefit. This finding has practical implications: for deployment in production systems, conservative exploration parameters may be preferable once sufficient initial data has been collected.

6.3 Ablation Study: Tradeoff Parameter Sensitivity

The trade-off parameter λ directly controls the balance between retrieval quality and computational efficiency in the reward function (Equation 4). Analysis of policy proportions across λ values (Figure 5) reveals algorithm-specific sensitivities to this parameter.

For Epsilon-Greedy, reducing λ from 0.9 to 0.1 causes a substantial policy shift: at $\lambda = 0.1$, the algorithm allocates approximately 90% selection probability to Hybrid ($\alpha = 0.75$) compared to minimal selection at $\lambda = 0.5$ or $\lambda = 0.9$. This dramatic shift reflects Epsilon-Greedy’s sensitivity to the reward signal. When latency is weakly penalized ($\lambda = 0.1$), the high recall of Hybrid ($\alpha = 0.75$) dominates the reward calculation despite its high latency, causing the algorithm to favor it. Conversely, at $\lambda = 0.9$, latency penalties dominate, driving selection toward faster arms like dense retrieval. This sensitivity can be viewed as both a strength (responsiveness to reward structure) and weakness (potential brittleness to reward function specification).

UCB and Thompson Sampling exhibit greater stability in policy proportions across λ values, though both algorithms still increase selection of higher-latency, higher-recall arms as λ decreases.

Examining Table 3, the performance implications become clear. At $\lambda = 0.1$, all algorithms achieve their highest recall scores: Epsilon-Greedy reaches 92.27% (vs 90.52% at $\lambda = 0.5$), UCB reaches 91.64% (vs 90.92%), and Thompson Sampling reaches 92.91% (vs 92.87%) on MS MARCO. However, these gains come with latency increases: Epsilon-Greedy’s latency rises to 11.28ms (from 6.90ms), indicating its policy shift toward Hybrid ($\alpha = 0.75$). Thompson Sampling maintains better balance, achieving near-optimal recall (92.91%) with moderate latency (10.52ms).

The relative insensitivity of Thompson Sampling’s recall scores to λ variations (92.91%, 92.87%,

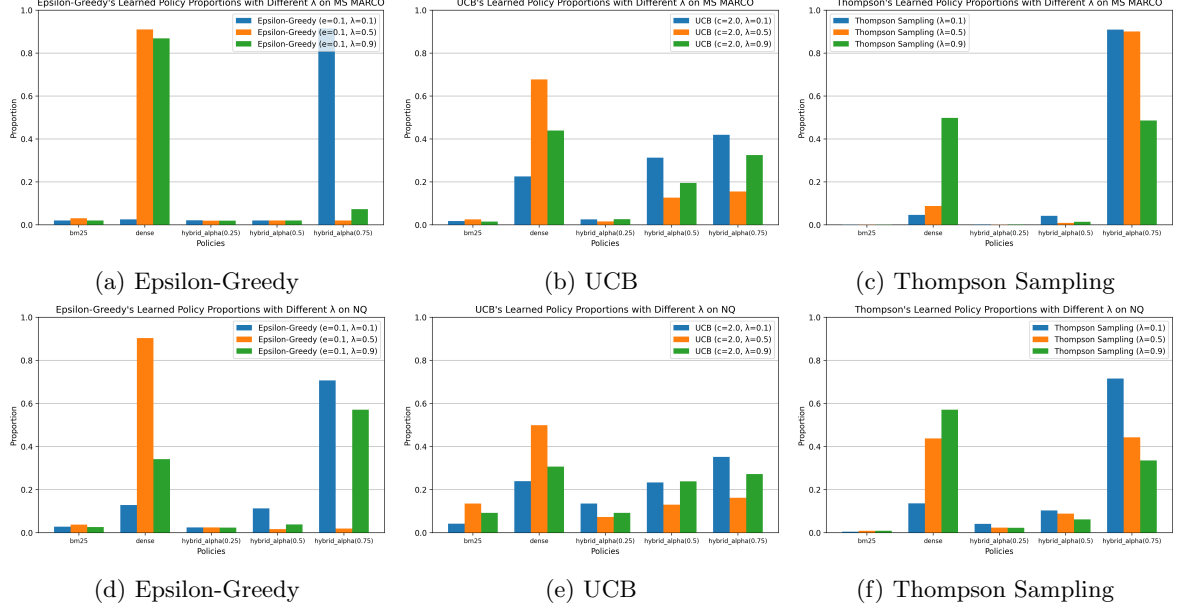


Figure 5: Policy proportions across λ values: Top row shows MS MARCO results for (a) Epsilon-Greedy, (b) UCB, and (c) Thompson Sampling. Bottom row shows Natural Questions results for the same algorithms.

Algorithm	λ	MS MARCO		Natural Questions (NQ)	
		Recall@10	Latency (ms)	Recall@10	Latency (ms)
Epsilon-Greedy ($\epsilon = 0.1$)	0.1	0.9227	11.28	0.7911	11.01
	0.5	0.9052	6.90	0.7715	6.73
	0.9	0.9082	7.22	0.7856	9.93
UCB ($c = 2.0$)	0.1	0.9164	10.66	0.7659	10.16
	0.5	0.9092	8.74	0.7530	8.20
	0.9	0.9144	9.51	0.7609	9.47
Thompson Sampling	0.1	0.9291	10.52	0.7906	11.40
	0.5	0.9287	10.80	0.7849	9.65
	0.9	0.9214	8.47	0.7830	8.80

Table 3: Sensitivity of MAB algorithms to trade-off parameter λ . Higher λ penalizes latency more heavily.

92.14% for $\lambda \in \{0.1, 0.5, 0.9\}$) while adapting latency appropriately (10.52ms, 10.80ms, 8.47ms) demonstrates its robustness. This suggests Thompson Sampling can maintain strong performance across different deployment scenarios with varying latency constraints without requiring extensive hyperparameter tuning — a significant practical advantage.

6.4 Error Analysis

The violin plots in Figure 6 provide insight into arm-specific performance characteristics and potential algorithm failure modes. For BM25 across all algorithms, the violin distributions exhibit bimodal or dumbbell shapes with high density near both 0 and 1, indicating that BM25 selections result in either very high or very low recall. This bimodality reflects BM25’s strength as a keyword-matching method: it excels for queries with precise keyword overlap between query and relevant documents but fails for queries requiring semantic understanding or synonym matching. Similarly, Hybrid ($\alpha = 0.25$), which emphasizes BM25 through its low α value, exhibits analogous bimodal patterns, though with slightly better concentration at high recall scores.

In contrast, dense retrieval, Hybrid ($\alpha = 0.5$), and Hybrid ($\alpha = 0.75$) display violin distributions with high density concentrated at high recall scores and thin tails at low scores. This indicates these

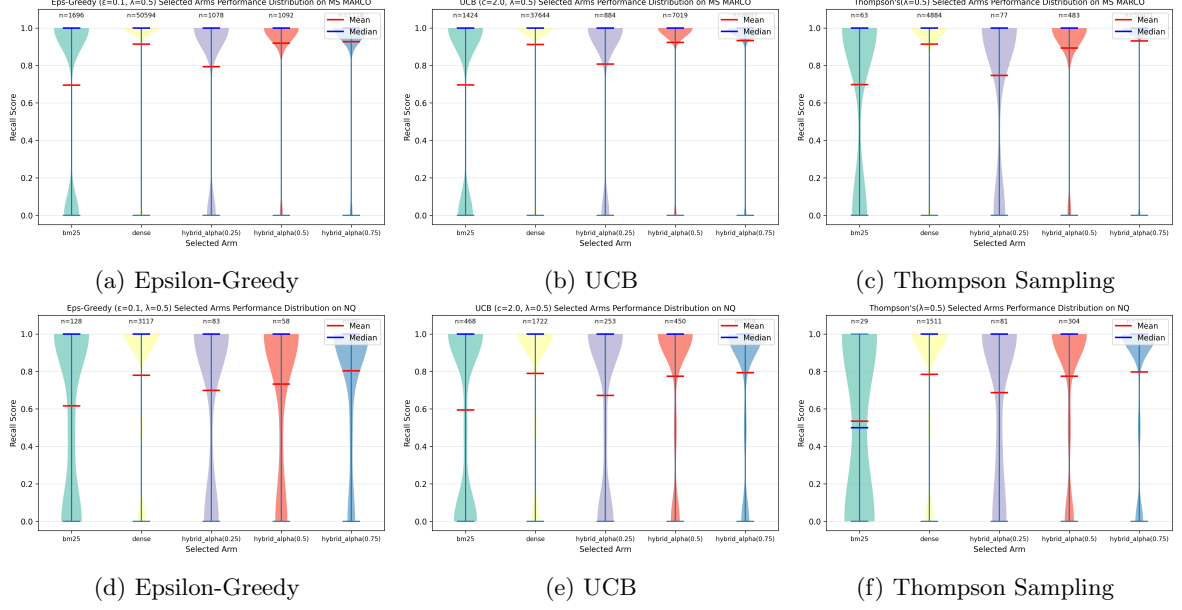


Figure 6: Recall score distributions by arm for baseline MAB configurations. Top: MS MARCO. Bottom: Natural Questions.

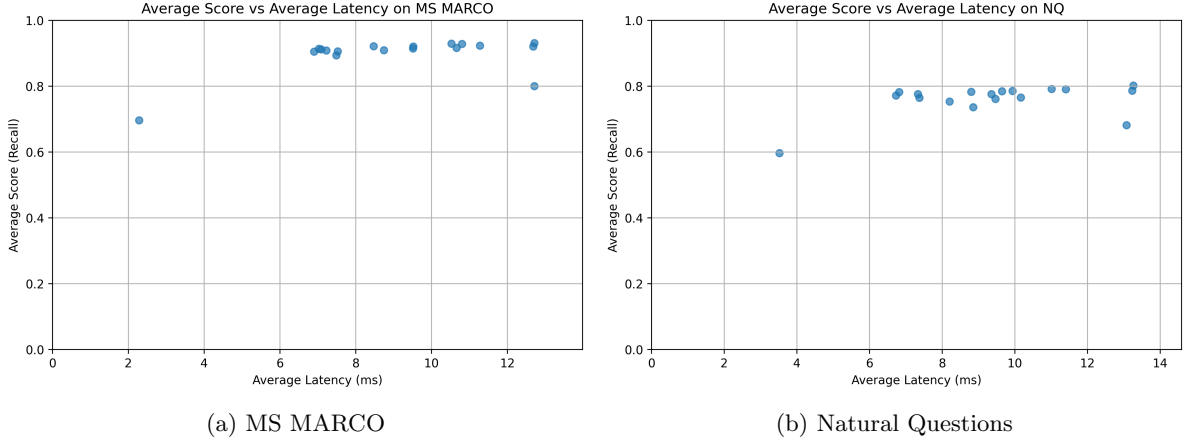


Figure 7: Average score vs. latency trade-off across all methods

arms consistently perform well when selected. The concentration of mass at high recall values validates the algorithms’ learned preferences for these arms.

Comparing algorithms, Epsilon-Greedy and UCB exhibit remarkably similar violin distributions across arms, corroborating earlier observations that these algorithms learn similar policies and achieve comparable performance. Both show frequent selection of arms that perform well (dense retrieval, Hybrid $\alpha = 0.75$) and occasional selection of BM25, likely during exploration phases.

Thompson Sampling presents an interesting anomaly on the NQ dataset (Figure 6f): the BM25 violin appears nearly rectangular, indicating uniform distribution of recall scores when BM25 is selected. This suggests high variance in BM25’s performance for the queries where Thompson Sampling chose it. However, inspection of the policy proportions reveals BM25 receives minimal selection probability (1%), indicating Thompson Sampling quickly learned to avoid this arm after initial exploration. The few remaining BM25 selections likely occur due to the probabilistic sampling process occasionally producing high uncertainty estimates, triggering exploratory pulls. Critically, this does not substantially impact Thompson Sampling’s overall performance, as evidenced by its superior recall and balanced latency.

The violin plots also reveal relatively few instances of arms producing very low recall scores when selected, suggesting all algorithms largely avoid catastrophic failures once past initial learning phases. This is encouraging for deployment: even with continuous exploration, the algorithms rarely make severely suboptimal decisions.

6.5 Limitations and Future Directions

The current study implements non-contextual bandits, which select arms based solely on empirical reward history without incorporating query-specific information. A promising extension involves implementing contextual bandits such as LinUCB, which incorporate query embeddings to guide arm selection. Contextual approaches would enable algorithms to learn query-dependent policies (e.g., selecting BM25 for keyword-focused queries and hybrid search for complex semantic queries), potentially yielding superior performance. However, contextual methods introduce additional computational overhead: query embeddings must be computed, and LinUCB requires matrix inversion operations that scale poorly with dimensionality. The performance-latency trade-off for contextual bandits warrants careful investigation.

Additionally, this study evaluates algorithms in an online learning setting where each query serves simultaneously as training data and test instance. While this paradigm is appropriate for the bandit framework, it differs from traditional offline evaluation where training and test sets are separated. Future work could investigate the algorithms’ generalization to unseen query distributions by training on one dataset or time period and evaluating on another.

Finally, the reward function employed (Equation 4) uses static normalization of latency based on running minimum and maximum values. Alternative formulations, such as percentile-based normalization or non-linear penalty functions, may provide better reward shaping. Investigating reward function design represents a valuable direction for improving algorithm performance.

7 Conclusion

This study has demonstrated that MAB algorithms can adaptively select retrieval strategies, achieving strong performance-latency balance without any hard-coded rules. These are probabilistic models that select arms probabilistically, receive a reward based on recall and latency and update its policy to improve future selections. Furthermore, this is an online learning algorithm meaning that no train-test split is needed. This is in contrast to Neural Networks which learn a weight matrix W and bias b during training and has to do evaluation offline using the test set. This online learning nature makes it extremely popular in applications such as recommendation systems or online advertising where data drift (change in distribution of data) could happen and the model needs to adapt quickly. With MAB, they are not required to be taken down for retraining or finetuning.

This study demonstrates that Multi-Armed Bandit algorithms enable adaptive retrieval strategy selection, achieving favorable performance-latency trade-offs without hard-coded rules. MAB algorithms operate as probabilistic models that select arms stochastically, receive rewards based on recall and latency, and update policies to improve future selections through online learning without requiring train-test splits. This contrasts with neural network approaches that learn fixed parameters during training and require offline evaluation on test sets.

Among the three algorithms evaluated, Thompson Sampling demonstrates superior performance, achieving recall scores approaching the best static baseline (Hybrid Search with $\alpha = 0.75$) while reducing latency by approximately 15%. Thompson Sampling’s probabilistic exploration through Bayesian uncertainty quantification provides natural regularization, yielding robust performance across diverse hyperparameter configurations. Epsilon-Greedy and UCB exhibit comparable performance to each other, both favoring dense retrieval, but fail to match Thompson Sampling’s efficiency.

Ablation studies reveal that lower exploration parameters (lower ϵ for Epsilon-Greedy, lower c for UCB) yield higher recall at modest latency costs, suggesting that optimal arms can be identified quickly in this setting and aggressive continued exploration provides diminishing returns. The reward trade-off parameter λ significantly influences policy selection, with lower values encouraging selection of higher-recall, higher-latency arms. Thompson Sampling demonstrates particular robustness to λ variations, maintaining strong performance across different latency constraint scenarios.

The online learning nature of MAB algorithms offers significant practical advantages for deployment in production systems. Unlike neural networks requiring retraining or fine-tuning when data distributions shift, MAB algorithms continuously adapt to changing query patterns. This property makes them particularly valuable for applications such as recommendation systems or online advertising where data drift is common and models must adapt rapidly.

While this investigation focuses on non-contextual bandits, future extensions incorporating query context through methods like LinUCB or neural contextual bandits could further improve performance by learning query-dependent policies. However, such approaches must carefully balance the additional computational cost of context processing against performance gains.

In summary, this study validates the application of Multi-Armed Bandit algorithms for adaptive retrieval, demonstrating that Thompson Sampling in particular can approach optimal static performance while maintaining computational efficiency through intelligent online learning.

References

- Payal Bajaj, Daniel Campos, Nick Craswell, Li Deng, Jianfeng Gao, Xiaodong Liu, Rangan Majumder, Andrew McNamara, Bhaskar Mitra, Tri Nguyen, et al. Ms marco: A human generated machine reading comprehension dataset. *arXiv preprint arXiv:1611.09268*, 2016.
- Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C. Park. Adaptive-rag: Learning to adapt retrieval-augmented large language models through question complexity, 2024. URL <https://arxiv.org/abs/2403.14403>.
- Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Matthew Kelcey, Jacob Devlin, Kenton Lee, Kristina N. Toutanova, Llion Jones, Ming-Wei Chang, Andrew Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. Natural questions: a benchmark for question answering research. *TACL*, 2019.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks, 2021. URL <https://arxiv.org/abs/2005.11401>.
- Xiaqiang Tang, Qiang Gao, Jian Li, Nan Du, Qi Li, and Sihong Xie. Mba-rag: a bandit approach for adaptive retrieval-augmented generation through question complexity, 2025. URL <https://arxiv.org/abs/2412.01572>.
- Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*, 2021. URL <https://openreview.net/forum?id=wCu6T5xFjeJ>.

8 Appendix

Algorithm 1 Epsilon-Greedy Training Process

```

1: Initialize:  $\epsilon$  (exploration rate),  $\lambda$  (latency penalty)
2: Initialize: For each arm  $a \in \{1, \dots, K\}$ :  $Q(a) \leftarrow 0$ ,  $N(a) \leftarrow 0$ 
3: Initialize:  $\text{lat}_{\min} \leftarrow \infty$ ,  $\text{lat}_{\max} \leftarrow -\infty$ 
4: Input: List of queries  $Queries$ 
5:
6: for each  $query \in Queries$  do
7:    $p \leftarrow \text{random\_uniform}(0, 1)$ 
8:   if  $p < \epsilon$  then
9:      $a_t \leftarrow \text{random\_arm}()$  ▷ Explore
10:  else
11:     $a_t \leftarrow \arg \max_a Q(a)$  ▷ Exploit
12:  end if
13:
14:   $\text{pids}_{\text{ret}}, \text{lat} \leftarrow \text{retrieve}(a_t, query)$  ▷ Get retrieved passage IDs and latency
15:   $\text{pids}_{\text{rel}} \leftarrow \text{get\_relevant\_pids}(query)$  ▷ Ground Truth pids
16:
17:   $\text{score} \leftarrow \text{Recall@k}(\text{pids}_{\text{ret}}, \text{pids}_{\text{rel}})$ 
18:  Update  $\text{lat}_{\min}, \text{lat}_{\max}$ 
19:   $\text{cost}_{\text{norm}} \leftarrow (\text{lat} - \text{lat}_{\min}) / (\text{lat}_{\max} - \text{lat}_{\min})$  ▷ Normalize latency to  $[0, 1]$ 
20:   $R_t \leftarrow \text{score} - \lambda \times \text{cost}_{\text{norm}}$  ▷ Compute Reward
21:
22:   $N(a_t) \leftarrow N(a_t) + 1$  ▷ Increment Pull Count
23:   $Q(a_t) \leftarrow Q(a_t) + \frac{1}{N(a_t)}(R_t - Q(a_t))$  ▷ Update Estimated Reward
24: end for

```

Algorithm 2 UCB Training Process

```

1: Initialize:  $c$  (exploration parameter),  $\lambda$  (latency penalty)
2: Initialize: For each arm  $a \in \{1, \dots, K\}$ :  $Q(a) \leftarrow 0$ ,  $N(a) \leftarrow 0$ 
3: Initialize:  $\text{lat}_{\min} \leftarrow \infty$ ,  $\text{lat}_{\max} \leftarrow -\infty$ ,  $t \leftarrow 0$ 
4: Input: List of queries  $Queries$ 
5:
6: for each  $query \in Queries$  do
7:    $t \leftarrow t + 1$  ▷ Increment global timestep counter
8:   if any  $N(a) = 0$  then ▷ Ensure each arm is pulled at least once for initialization
9:      $a_t \leftarrow \text{first arm with } N(a) = 0$  ▷ Pull any untried arm
10:  else ▷ All arms tried at least once, use UCB selection
11:     $a_t \leftarrow \arg \max_a \left( Q(a) + c \sqrt{\frac{\ln(t)}{N(a)}} \right)$  ▷ Exploitation + Exploration
12:  end if
13:
14:   $\text{pids}_{\text{ret}}, \text{lat} \leftarrow \text{retrieve}(a_t, query)$ 
15:   $\text{pids}_{\text{rel}} \leftarrow \text{get\_relevant\_pids}(query)$ 
16:
17:   $\text{score} \leftarrow \text{Recall@k}(\text{pids}_{\text{ret}}, \text{pids}_{\text{rel}})$ 
18:  Update  $\text{lat}_{\min}, \text{lat}_{\max}$ 
19:   $\text{cost}_{\text{norm}} \leftarrow (\text{lat} - \text{lat}_{\min}) / (\text{lat}_{\max} - \text{lat}_{\min})$ 
20:   $R_t \leftarrow \text{score} - \lambda \times \text{cost}_{\text{norm}}$ 
21:
22:   $N(a_t) \leftarrow N(a_t) + 1$ 
23:   $Q(a_t) \leftarrow Q(a_t) + \frac{1}{N(a_t)}(R_t - Q(a_t))$ 
24: end for

```

Algorithm 3 Empirical Gaussian Thompson Sampling Training Process

```
1: Initialize:  $\lambda$  (latency penalty)
2: Initialize: For each arm  $a \in \{1, \dots, K\}$ :  $Q(a) \leftarrow 0$ ,  $N(a) \leftarrow 0$ ,  $\text{SumSq}(a) \leftarrow 0$ 
3: Initialize:  $\text{lat}_{\min} \leftarrow \infty$ ,  $\text{lat}_{\max} \leftarrow -\infty$ 
4: Input: List of queries  $Queries$ 
5:
6: for each  $query \in Queries$  do
7:   for each arm  $a \in \{1, \dots, K\}$  do
8:     if  $N(a) \leq 1$  then                                     ▷ Not enough data to estimate variance reliably
9:        $\mu_a \leftarrow Q(a)$                                        ▷ Use current mean estimate
10:       $\text{SE}_a \leftarrow 1.0$                                        ▷ High initial uncertainty to encourage exploration
11:     else                                                       ▷ Sufficient data to compute empirical statistics
12:        $\mu_a \leftarrow Q(a)$                                        ▷ Empirical mean of rewards
13:        $\hat{\sigma}_a^2 \leftarrow (\text{SumSq}(a)/N(a)) - \mu_a^2$            ▷ Empirical variance:  $E[X^2] - (E[X])^2$ 
14:        $\hat{\sigma}_a^2 \leftarrow \max(\hat{\sigma}_a^2, 10^{-6})$                ▷ Clamp to avoid numerical issues
15:        $\text{SE}_a \leftarrow \sqrt{\hat{\sigma}_a^2/N(a)}$                        ▷ Standard Error of Mean
16:     end if
17:      $S_a \sim \mathcal{N}(\mu_a, \text{SE}_a^2)$                                ▷ Sample from belief distribution
18:   end for
19:    $a_t \leftarrow \arg \max_a (S_a)$                                ▷ Select arm with highest sampled value
20:
21:    $\text{pids}_{\text{ret}}, \text{lat} \leftarrow \text{retrieve}(a_t, query)$ 
22:    $\text{pids}_{\text{rel}} \leftarrow \text{get\_relevant\_pids}(query)$ 
23:
24:    $\text{score} \leftarrow \text{Recall@k}(\text{pids}_{\text{ret}}, \text{pids}_{\text{rel}})$ 
25:   Update  $\text{lat}_{\min}, \text{lat}_{\max}$ 
26:    $\text{cost}_{\text{norm}} \leftarrow (\text{lat} - \text{lat}_{\min})/(\text{lat}_{\max} - \text{lat}_{\min})$ 
27:    $R_t \leftarrow \text{score} - \lambda \times \text{cost}_{\text{norm}}$ 
28:
29:    $N(a_t) \leftarrow N(a_t) + 1$ 
30:    $\text{SumSq}(a_t) \leftarrow \text{SumSq}(a_t) + R_t^2$ 
31:    $Q(a_t) \leftarrow Q(a_t) + \frac{1}{N(a_t)}(R_t - Q(a_t))$ 
32: end for
```
